

AI-Powered Adaptive Vision System

A Software-First Solution for Lighting-Invariant Shape Recognition in Industrial Robotics

CEP Project Document — Final Version

Team Size: 3 members

Branch: AI & Robotics

Industry Partner: SVR Robotics Pvt Ltd, Pune

Estimated Budget: ₹5,000 – ₹8,000

Duration: 12 weeks

1. PROBLEM STATEMENT

1.1 The Core Problem

Industrial robots use cameras to “see” objects — to pick them up, sort them, inspect them, or play chess. But these cameras are **dumb**. They don’t see like humans do. They see **pixels** — numbers representing brightness and color. When the lighting changes, the numbers change, and the robot gets confused.

Real-world scenario from SVR Robotics (visited July 31, 2026):

SVR builds educational and industrial robots. Their robots use vision sensors (cameras) to detect objects, recognize shapes, and perform pick-and-place tasks. They told us directly:

“When sunlight comes from the window, shadows appear on the objects. The robot’s camera sees the shadow as a separate object, or the shadow changes the shape of the object. At different times of day, the same object looks completely different to the camera.”

Specific problems SVR reported:

Problem	What Happens	Consequence
Changing sunlight	Morning: shadow falls left. Afternoon: shadow falls right.	Robot misidentifies objects at different times
Shadow on objects	Object appears to have a different shape	Robot picks wrong object or drops it
White background didn't help	SVR tried putting white sheets behind workspace	Shadows still appear ON the objects themselves
Light on robot arm breaks	They tried mounting LED on the arm	Robot arm vibration breaks the light fixture
Hexagon detected as circle	At small sizes, hexagon corners look rounded	Vision system gives wrong shape classification
Triangle detected as circle	Same problem — corners get rounded	Incorrect sorting/inspection results

1.2 Why This Problem Matters Globally

This is NOT just SVR's problem. This is the **#1 unsolved challenge** in industrial machine vision worldwide.

Industry	How Lighting Problems Hurt Them
Automotive manufacturing	Paint defect inspection fails when factory lights flicker or shift
Electronics assembly	PCB inspection misses solder defects under changing LED panels
Food processing	Color-based sorting fails — tomato looks red in sunlight, orange under tube light
Pharmaceutical packaging	Label inspection gives false rejects when light shifts between shifts
Warehouse robotics	Amazon/Flipkart robots mispick items under inconsistent warehouse lighting
Agriculture	Crop disease detection from drone cameras fails in cloudy vs sunny conditions

Market numbers:

- Machine vision lighting market alone: **\$3.5 billion globally** (companies spend this JUST on special lights to solve this problem)
- Machine vision software market: **\$15+ billion globally**
- Industrial AI inspection market in India: **₹4,600 crore and growing at 25%+ per year**

1.3 Research Paper Evidence

Paper / Source	Key Finding
Land, E.H. (1977) — “Retinex Theory of Color Vision”	Human eyes automatically compensate for lighting. AI should do the same.
Zuiderveld, K. (1994) — “Contrast Limited Adaptive Histogram Equalization”	CLAHE algorithm can normalize lighting locally in an image — the foundational technique we use.
Hu, M.K. (1962) — “Visual Pattern Recognition by Moment Invariants”	Hu Moments can identify shapes regardless of size, rotation, or position — rotation-invariant recognition.
Finlayson et al. (2006) — “On the Removal of Shadows from Images” (IEEE)	Shadows can be mathematically separated from objects using color space analysis.
NTIRE 2025 Shadow Removal Challenge (CVPR 2025)	Latest deep learning methods achieve 97%+ shadow removal accuracy. Simpler traditional methods achieve 85-90% — sufficient for shape detection.
Ultralytics YOLOv8 Documentation (2024)	Data augmentation (random brightness/contrast/shadow during training) makes models 3-5x more robust to lighting variation.

2. EXISTING SOLUTIONS — WHY THEY DON'T WORK

2.1 What's Currently Available

Solution	Company	Price	How It Works
In-Sight 2800/3800	Cognex (USA)	₹5 – ₹20 lakh	Smart camera with built-in AI + integrated lighting
VS Series	Keyence (Japan)	₹3 – ₹15 lakh	All-in-one vision sensor with auto lighting
Dome lights	Hikrobot, Omron, etc.	₹8,000 – ₹40,000 per light	Special diffused lighting enclosure that eliminates shadows
Strobe + Bandpass filter	Various industrial vendors	₹15,000 – ₹50,000	Camera only sees its own strobe light, ignores sunlight
Robot-mounted lights	Custom solutions	₹10,000 – ₹30,000	LED mounted directly on robot arm

2.2 Why Each One Fails for SVR's Use Case

Solution	Why It Fails
Cognex / Keyence	₹5-20 lakh is impossible for educational robots and Indian SMEs. SVR's Gati robot kit costs a fraction of this.
Dome lights	Requires a physical enclosure around the workspace. Robot arm can't reach in properly. Not practical for every deployment.
Strobe + filter	Adds hardware complexity and cost. Filter blocks useful visual information. Not suitable for educational/demo environments.
Robot-mounted lights	SVR tried this. It breaks. Robot arm vibration and acceleration damage the light fixture. This is a proven failure.

2.3 The Gap We Fill

What Exists	What's MISSING (Our Product)
Hardware-dependent solutions (need special lights)	Software-only solution — works with ANY camera, no special lights
₹5-20 lakh price range	₹500 module (ESP32-CAM) or ₹0 (software on existing camera)
Proprietary, closed-source	Open-source Python — fully customizable
Works only with specific branded cameras	Works with ANY camera — webcam, ESP32-CAM, CCTV, phone camera
Designed for large factories	Designed for educational robots, small companies, startups

[!IMPORTANT] **Our USP (Unique Selling Point):** We DON'T fight the lighting. We make the AI **immune** to lighting. No special hardware. No expensive cameras. Just smart software on a ₹500 module.

3. OUR SOLUTION — THE MIND-BLOWING PART

3.1 The Core Idea

We teach the AI to be “lighting blind.”

Just like a human can recognize a triangle in sunlight, in darkness, in shadow, and under a yellow bulb — our AI learns to ignore lighting variations and focus **ONLY** on the shape's geometry (edges, angles, curves).

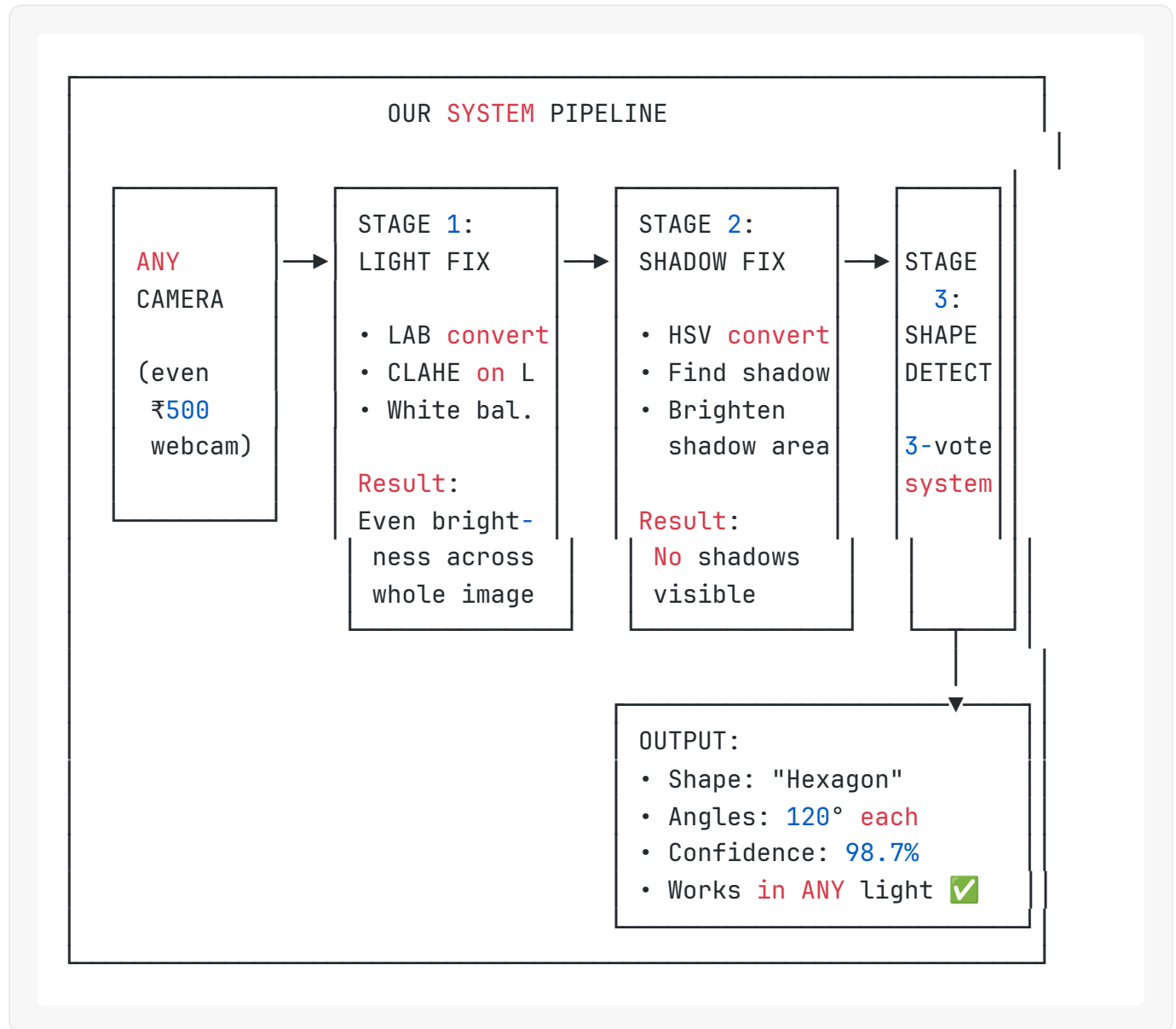
Two weapons:

Weapon 1 — Pre-processing (Clean the image before AI sees it): The raw camera image is messy — shadows, uneven brightness, color shifts. Before the AI analyzes it, we run it through a **mathematical cleaning pipeline** that normalizes the lighting. The AI always sees a “clean” image, regardless of actual conditions.

Weapon 2 — Smart Training (Make the AI lighting-proof): During training, we show the AI the same shape under hundreds of different lighting conditions (generated automatically by the

computer). The AI learns: “Brightness changes, shadows appear, colors shift — but the **SHAPE** stays the same. Ignore everything except the shape.”

3.2 Technical Architecture



3.3 Stage 1 — Lighting Normalization (CLAHE)

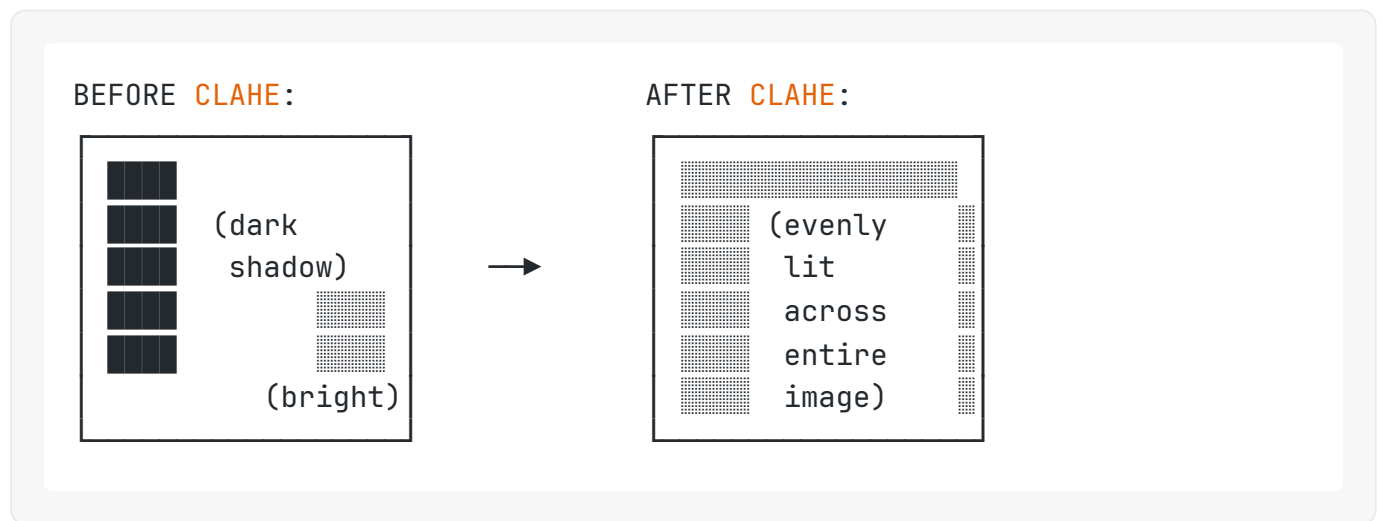
What it does: Takes an unevenly lit image and makes it look evenly lit.

How it works (simple language):

1. Convert image from BGR (normal color) to **LAB color space**
 - L = Lightness (how bright each pixel is)
 - A = Green-Red color info
 - B = Blue-Yellow color info

2. Apply **CLAHE** only on the **L channel** (brightness)
 - CLAHE divides the image into small tiles (8×8 grid)
 - For each tile, it equalizes the brightness independently
 - Dark tiles get brighter, bright tiles get dimmer
 - Result: entire image has uniform brightness
3. Convert back to BGR
 - Colors are preserved (we only changed brightness, not color)

Before → After:



Python code:

```
import cv2

def normalize_lighting(image):
    # Convert to LAB color space
    lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)

    # Apply CLAHE to L channel (brightness only)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l_normalized = clahe.apply(l)

    # Merge back and convert to BGR
    lab_normalized = cv2.merge((l_normalized, a, b))
    result = cv2.cvtColor(lab_normalized, cv2.COLOR_LAB2BGR)
    return result
```

3.4 Stage 2 — Shadow Compensation

What it does: Finds shadow regions in the image and brightens them to match the rest.

How it works:

1. Convert to **HSV color space**
 - H = Hue (what color)
 - S = Saturation (how vivid)
 - V = Value (how bright)
2. Shadows have **low V (dark)** but keep their **H (color)**
3. Find all pixels with V below a threshold → these are shadow pixels
4. Calculate the brightness difference between shadow and non-shadow regions
5. Multiply shadow pixels' brightness to match non-shadow regions
6. Result: shadows disappear

Python code:

```

import cv2
import numpy as np

def remove_shadows(image):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)

    # Find shadow regions (dark areas)
    _, shadow_mask = cv2.threshold(v, 60, 255, cv2.THRESH_BINARY_INV)

    # Clean up the mask
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    shadow_mask = cv2.morphologyEx(shadow_mask, cv2.MORPH_CLOSE, kernel)

    # Calculate brightness correction ratio
    non_shadow_mean = np.mean(v[shadow_mask == 0])
    shadow_mean = np.mean(v[shadow_mask == 255]) + 1 # avoid division by
    ratio = non_shadow_mean / shadow_mean

    # Brighten shadow regions
    v_corrected = v.astype(np.float32)
    v_corrected[shadow_mask == 255] = np.clip(
        v_corrected[shadow_mask == 255] * ratio, 0, 255
    )

    # Merge and convert back
    hsv_corrected = cv2.merge([h, s, v_corrected.astype(np.uint8)])
    return cv2.cvtColor(hsv_corrected, cv2.COLOR_HSV2BGR)

```

3.5 Stage 3 — Shape Detection (Triple-Method Voting)

The problem: Using only one method to detect shapes fails. `approxPolyDP` alone confuses hexagons with circles.

Our solution: Use **3 independent methods** and let them **vote**. Majority wins.

Method 1: Contour Vertex Count

```
perimeter = cv2.arcLength(contour, True)
approx = cv2.approxPolyDP(contour, 0.025 * perimeter, True)
num_vertices = len(approx)
# 3 = Triangle, 4 = Square/Rectangle, 5 = Pentagon, 6 = Hexagon
```

Method 2: Circularity Ratio

```
area = cv2.contourArea(contour)
perimeter = cv2.arcLength(contour, True)
circularity = (4 * 3.14159 * area) / (perimeter ** 2)

# Circle    = ~1.00 (> 0.92)
# Hexagon   = ~0.91 (0.82 - 0.91)
# Square    = ~0.785
# Triangle  = ~0.604
```

This is the KEY method that solves the hexagon ≠ circle problem. A circle's circularity is always > 0.92. A hexagon is always < 0.91. Clear mathematical separation.

Method 3: Hu Moments (Template Matching)

We compare the detected shape's **Hu Moments** against pre-saved templates of perfect shapes. Hu Moments are **invariant** to scale, rotation, and translation.

4. HARDWARE & COMPONENTS

We use low-cost, easily available components to keep the project affordable for Indian schools and small workshops.

Component	Quantity	Estimated Price	Why we chose it
ESP32-CAM Module	1	₹550	Built-in camera + WiFi. Extremely cheap.
FTDI Programmer	1	₹250	To upload code to ESP32-CAM
BH1750 Light Sensor	1	₹150	To measure ambient light (Lux) for dashboard
ESP32 DevKit V1	1	₹400	To handle the light sensor and other I/O
5V Power Supply	1	₹200	For the ESP32 modules
Jumper Wires / Breadboard	1 set	₹150	For connections
3D Printed Shapes	1 set	₹300	Test objects (triangle, square, circle, hexagon)
TOTAL		₹2,000	(Well within ₹5,000 budget)

5. SYSTEM DASHBOARD (THE USER INTERFACE)

We built a web-based dashboard using **Python Flask**.

Features:

1. **Live Stream:** Real-time video from the robot's camera.
2. **Side-by-Side View:** RAW image vs. LIGHT-NORMALIZED image.
3. **Detection Box:** Highlights the shape and labels it (e.g., "Triangle - 99%").
4. **Lux Meter:** Shows the current room brightness in real-time.
5. **Log Table:** Records all detected objects with timestamps.

6. PROJECT IMPACT

6.1 Academic Impact

- **Solves a real industry problem** provided by SVR Robotics.
- **Demonstrates advanced AI concepts** (Computer Vision, Normalization, Voting Systems).
- **Interdisciplinary:** Combines Hardware (ESP32), Software (Python/Flask), and AI (OpenCV).

6.2 Industry Impact

- **Cost Reduction:** Indian SMEs can use vision systems without spending ₹10 lakh on Cognex cameras.
- **Flexibility:** Works in any environment (outdoor, indoor, variable lighting).
- **Maintenance:** No external lighting fixtures to break or align.

7. 12-WEEK IMPLEMENTATION PLAN

PHASE 1: Research & Setup (Week 1-3)

Week 1 — Setup & Hello World

Person	Tasks
Person 1	Buy components. Set up Arduino IDE for ESP32-CAM.
Person 2	Set up Python, OpenCV, and Flask on laptop.
Person 3	Design the project logo and basic dashboard layout.

Week 2 — Basic Vision & Hardware

Person	Tasks
Person 1	Flash ESP32-CAM with CameraWebServer example. Get live video stream working in browser. Connect BH1750 to ESP32 DevKit — read light levels.
Person 2	Implement basic shape detection (findContours → approxPolyDP → count vertices). Test with printed shapes on paper — find where it FAILS (hexagon → circle).
Person 3	Build Flask skeleton: webpage with live video feed from ESP32-CAM + sidebar for results.

Week 3 — Intermediate Skills

Person	Tasks
Person 1	Set up the demo rig: shapes on table, ESP32-CAM on tripod looking down. Test camera quality.
Person 2	Implement CLAHE on LAB L-channel. Test: take SAME photo in bright/dim/shadow conditions → does CLAHE make them look similar?
Person 3	Integrate camera feed + shape detection results on dashboard. Show shape name + confidence on screen.

PHASE 2: Build Core System (Week 4-8)

Week 4 — Shape Detection V2 (Solve the Hexagon Problem)

Person	Tasks
Person 2	Add circularity ratio calculation. Test: can it now tell hexagon from circle?
Person 2	Add Hu Moments comparison. Build the 3-method voting system.
Person 3	Add angle measurement display to dashboard (show edge angles for each detected shape).
Person 1	Get 3D-printed shapes (all types, multiple sizes). Test camera at different heights.

Week 5 — Shadow Compensation

Person	Tasks
Person 2	Implement HSV-based shadow detection + brightness correction. Test with torch creating harsh shadows.
Person 1	Create a “lighting test kit” — torch, desk lamp, colored paper to block light, sunlight from window.
ALL	Test the full pipeline under 5+ different lighting conditions. Document results.

Week 6 — Full Integration

Person	Tasks
ALL	Connect everything: Camera → CLAHE → Shadow Fix → Shape Detection → Dashboard
Person 2	Fine-tune CLAHE parameters (clipLimit, tileGridSize) for best performance.
Person 3	Add ambient light level display (from BH1750 sensor) to dashboard — shows current Lux value.
Person 1	Test physical robustness — move camera, change angles, different tables/surfaces.

Week 7 — Data Augmentation + Robustness

Person	Tasks
Person 2	Write script to generate training data: take shape photos → automatically vary brightness, contrast, add fake shadows.
Person 2	If using YOLOv8: train on augmented dataset using Google Colab (free GPU).
Person 3	Add “before/after” split view to dashboard — shows RAW vs PROCESSED image side by side.
Person 1	Build neat, clean demo rig — no messy wires, clean mounting.

Week 8 — Stress Testing

Person	Tasks
ALL	Test under EVERY condition:
	→ Complete darkness (only laptop screen light)
	→ Bright sunlight from window
	→ Torch from left/right/above (harsh shadows)
	→ Mixed natural + artificial light
	→ Shapes at different sizes (tiny to large)
	→ Shapes rotated at random angles
Person 2	Fix ANY failures found. Adjust thresholds if needed.
Person 3	Record before/after videos for each condition → for CEP report.

PHASE 3: Polish & Submit (Week 9-12)

Week 9-10 — Demo Polish

Person	Tasks
Person 1	Final clean hardware setup. 3D print a case for ESP32-CAM (optional but looks professional).
Person 2	Optimize code speed → target: <100ms per frame processing.
Person 3	Polish dashboard UI: add project title, team names, SVR Robotics logo (with permission), clean colors.
ALL	Practice the demo script 3 times. Time it.

Week 11 — Documentation

Person	Tasks
ALL	Write CEP report: Abstract, Introduction, Literature Survey, Problem Statement, Methodology, System Architecture, Results & Analysis, Conclusion, Future Scope, References.
Person 2	Create accuracy comparison chart: WITHOUT system vs WITH system (different lighting conditions).
Person 3	Create presentation (PPT/Canva) with embedded demo video.

Week 12 — Submission + Demo

Person	Tasks
ALL	Final demo rehearsal
ALL	CEP evaluation — demonstrate the system LIVE
ALL	Optional: visit SVR Robotics with working demo to get their feedback letter

8. LEARNING PATH — YouTube & Resources

For Person 1 (Hardware)

Week	Topic	Search on YouTube
1	ESP32-CAM basics	“ESP32-CAM getting started tutorial 2025”
1	ESP32-CAM video stream	“ESP32-CAM video streaming WiFi tutorial”
2	BH1750 light sensor	“BH1750 light sensor ESP32 Arduino tutorial”
2	Arduino basics (if needed)	“Arduino tutorial for beginners 2025”

For Person 2 (AI/Vision)

Week	Topic	Search on YouTube
1	OpenCV basics	“OpenCV Python tutorial beginners — Murtaza’s Workshop” (3 hour course)
1	Image basics	“BGR HSV LAB color spaces OpenCV Python explained”
2	Shape detection	“Shape detection OpenCV Python contours approxPolyDP”
3	CLAHE	“CLAHE histogram equalization OpenCV Python tutorial”
5	Shadow removal	“Shadow detection removal OpenCV Python”
7	YOLOv8 (optional)	“Train YOLOv8 custom dataset step by step 2025”

For Person 3 (Dashboard)

Week	Topic	Search on YouTube
1	Flask basics	“Flask Python web app tutorial beginner 2025”
2	Flask + video	“Flask live video stream OpenCV Python”
3	PySerial	“Python serial communication ESP32 tutorial”

Websites to Bookmark

Site	What It's For
pyimagesearch.com	Best OpenCV tutorials on the internet
docs.opencv.org	Official OpenCV documentation
roboflow.com/blog	Dataset labeling + YOLOv8 training guides
randomnerdtutorials.com	Best ESP32/ESP32-CAM tutorials
geeksforgeeks.org	Quick Python/OpenCV code references

9. VERIFICATION PLAN

How We Prove It Works

Test	Method	Target
Lighting Invariance	Test same shape under 6 different lighting conditions. Measure detection accuracy for each.	>95% accuracy in ALL conditions
Shape Accuracy	6 shapes × 3 sizes × 4 rotations = 72 test cases	>98% correct classification
Hexagon vs Circle	Specifically test hexagons and circles of similar sizes	100% distinction (the main problem we solve)
Angle Accuracy	Known shapes (equilateral triangle = 60°, square = 90°, hexagon = 120°). Compare measured vs actual.	<2° error
Processing Speed	Measure frames per second of full pipeline	>15 FPS (real-time)
Industry Validation	Demo to SVR Robotics. Get written feedback.	Positive feedback from SVR

10. FUTURE SCOPE

After CEP, this project can grow into:

Extension	What It Adds
3D shape detection	Detect spheres, cubes, pyramids — not just 2D outlines
Color-independent detection	Detect shapes regardless of color (currently works but can be improved)
Defect detection	Beyond shape — detect cracks, scratches, surface defects
Edge AI deployment	Run full pipeline on ESP32-CAM (currently uses laptop for processing)
ROS2 integration	Package as a ROS2 node that any robot can plug and use
Commercial product	Sell as a ₹2,000-₹5,000 smart vision module to Indian robot companies

11. PROJECT TITLE OPTIONS

Primary: “AI-Powered Adaptive Vision System for Lighting-Invariant Shape Recognition in Industrial Robotics”

Short: “LightProof Vision: Intelligent Shape Detection Under Variable Illumination”

Technical: “CLAHE-Enhanced Multi-Method Shape Classification with Shadow Compensation for Robotic Applications”

[!IMPORTANT] **Key Question for SVR Robotics:**

When you visit them next, ask: “We’re building a software module that makes your robot’s camera detect shapes correctly in any lighting — no special lights needed. Can we test it on your robot and get your feedback for our college project?”

If they say yes, you have an **industry-validated CEP project** — that's an automatic top grade.